The *thread_indent* function provides a way to better organise the printed results. It takes one argument—an indentation delta, which indicates how many spaces to add or remove from a thread's 'indentation counter'. It then returns a string with some generic trace data, along with an appropriate number of indentation spaces.

After running the script, when you create a file (from another terminal window, on ext3 formatted filesystem), you'll get the following output:

```
  0 touch(1546): -> ext3_permission
1792 touch(1546): <- ext3_permission
  0 touch(1546): -> ext3_permission
 91 touch(1546): <- ext3_permission
  0 touch(1546): -> ext3_lookup
 66 touch(1546):  -> ext3_find_entry
123 touch(1546):   -> ext3_getblk
171 touch(1546):    -> ext3_get_blocks_handle
214 touch(1546):     -> ext3_block_to_path
250 touch(1546):     <- ext3_block_to_path
  ..............
  ..............
```

Similarly, to trace all functions in a program, run the following command:

```
$ stap -e 'probe process("<program_path>").function("*") { log(pp()) }' -c
<program_path>
```

You'll need the debuginfo for the package that provides the program. For example, for */bin/ls*, you'll need the *debuginfo* of *coreutils*.

Kernel statement

You can put a probe on a specific line number in the kernel code. With the following code, you can probe when line number 836 of *fs/open.c* is executed, and then print the local variables of the running function.

```
probe kernel.statement("*@fs/open.c:836") {
        printf("local variables %s   \n", $$locals);
}
```

You'll get an output similar to what's shown below:

```
local variables inode=0xffff880086c6a7b0 error=0x0
```

Guru mode

Guru mode allows you to run the script without any memory and data protection, which is normally provided by the SystemTap access restrictions. It allows you to change things, rather than just observing. To run the script in guru mode, you need to run it with the *-g* option. You can then modify any data in the function, at a memory address, etc. You can add custom C code anywhere. However, note that you could easily mess something up and crash the kernel, so be very cautious when you use this mode.

Embedded C functions

General syntax:

```
function <name>:<type> ( <arg1>:<type>, ... ) %{ <C_stmts> %}
```

Embedded C code is permitted in a function body. In that case, the script language body is replaced entirely by a piece of C code enclosed between %{ and %} markers. We'll pass the arguments to this function, and will get a return value. To do this communication SystemTap implements the *THIS* structure. The *THIS* structure is dynamic, and contains the arguments you specify as variables, and a field called __retvalue, which is the value that will be returned to the call site.

Let's try to understand this, using an example from the Tapset library. To get the nice value of a process, just call *task_nice* from your stap script. Internally, in */usr/share/systemtap/tapset/task.stp*, this function is defined as:

```
// Return the nice value of the given task
function task_nice:long (task:long) %{ /* pure */
    struct task_struct *t = (struct task_struct *)(long)THIS->task;
    THIS->__retvalue = kread(&(t->static_prio)) - MAX_RT_PRIO - 20;
    CATCH_DEREF_FAULT();
%}
```

The function takes a long argument (*task*), and returns a long (nice value). Typecast the argument to *task_struct*, and later return the nice value by setting *THIS->__retvalue*.

Fault injection

As you can probe any kernel function and modify kernel variables, you can use these features to inject errors. Let's say you want to return an 'Invalid argument' error when a directory with a certain name is created.

From the kernel source:

```
SYSCALL_DEFINE2(mkdir, const char __user *, pathname, int, mode)
{
        return sys_mkdirat(AT_FDCWD, pathname, mode);
}
```

Now, let's put a probe when *sys_mkdir* returns and modify the return value if the directory name matches "mydir":

```
probe kernel.function("sys_mkdir").return {
        if (isinstr(user_string($pathname), "mydir")) {
                $return = -22;
        }
}
```